INNER JOIN – JUNÇÃO INTERNA (O "quem comeu?")

// para de fazer piada, Otávio, te acalma.

Imaginem que vocês são os donos do restaurante e quer saber quais clientes estão sentados comendo agora.

- Você só quer ver o nome do cliente SE ele tiver um prato na mesa (pedido).
- Se o Miguel entrou no restaurante mas ainda não pediu nada, ele não aparece.
- Se houver um pedido perdido na cozinha sem nome de cliente, ele também não aparece.

Resultado: Só aparece o "match" perfeito entre Cliente e Pedido, ambos presentes.

LEFT JOIN – JUNÇÃO À ESQUERDA (A "lista de presença")

Agora imaginem que vocês querem ver a lista de todos os clientes que entraram na sua loja hoje, independentemente de terem gastado dinheiro ou não.

- Você lista TODOS os nomes da tabela da esquerda (Clientes).
- Se o cliente comprou, o nome do pedido aparece do lado.
- Se o cliente (como a LarissaProPlayer) só entrou para olhar e não pediu nada, o nome dela aparece na lista, mas no lugar do pedido fica escrito NULL (que significa: "veio, mas não pediu nada").

Resultado: Você não perde ninguém da sua lista principal. Ou seja, todos os clientes aparecem. Difícil?

Sim, inicialmente confuso, mas é isso aí, vamos caminhar e usar exemplos. Façam os próximos exercícios e vejam na prática:

EXERCÍCIO 1: JOGADORES COM INNER E LEFT

Para quem vai fazer no site (não recomendo, somente utilizem caso o xampp e o BD não tenha como instalar no pc de vocês):

Digite um título...

SQL

```
Código
-- Criando a estrutura
CREATE TABLE Jogadores (
    id_aluno INT PRIMARY KEY,
    nick_name VARCHAR(50)
);

CREATE TABLE Skins (
    id_skin INT PRIMARY KEY,
    nome_skin VARCHAR(50),
    id_dono INT
);

-- Inserindo os dados de teste
INSERT INTO Jogadores VALUES (1, 'NinjaPro'), (2, 'CatPlayer'), (3, 'Ghost');
INSERT INTO Skins VALUES (10, 'Capa de Fogo', 1), (11, 'Espada Laser', 2);


SELECT Jogadores.nick_name, Skins.nome_skin
FROM Jogadores
INNER JOIN Skins ON Jogadores.id_aluno = Skins.id_dono;


SELECT Jogadores.nick_name, Skins.nome_skin
FROM Jogadores
LEFT JOIN Skins ON Jogadores.id_aluno = Skins.id_dono;

```

Para quem vai fazer assim no lab usando o Workbench:

```sql
-- Exercício 1
-- Criando a estrutura
CREATE TABLE Jogadores (
    id_aluno INT PRIMARY KEY,
    nick_name VARCHAR(50)
);

CREATE TABLE Skins (
    id_skin INT PRIMARY KEY,
    nome_skin VARCHAR(50),
    id_dono INT
);

select * from jogadores;

-- Inserindo os dados de teste
INSERT INTO Jogadores VALUES (1, 'NinjaPro'), (2, 'CatPlayer'), (3, 'Ghost');
INSERT INTO Skins VALUES (10, 'Capa de Fogo', 1), (11, 'Espada Laser', 2);


SELECT Jogadores.nick_name, Skins.nome_skin
FROM Jogadores
INNER JOIN Skins ON Jogadores.id_aluno = Skins.id_dono;

```

```sql
SELECT Jogadores.nick_name, Skins.nome_skin
FROM Jogadores
LEFT JOIN Skins ON Jogadores.id_aluno = Skins.id_dono;

```

EXPLICANDO O QUE SIGNIFICA PALAVRA POR PALAVRA DENTRO DO INNER JOIN

Linha 19: SELECT Jogadores.nick_name, Skins.nome_skin

O SELECT é o comando de "busca". Aqui vocês estão dizendo ao banco: "quero que você me mostre apenas duas colunas na tela final: o apelido do jogador (nick_name) e o nome da roupinha dele (nome_skin)".

O ponto (.) serve para indicar quem é o dono da coluna. Como temos duas tabelas, precisamos avisar que o **nick_name vem da tabela Jogadores** e o **nome_skin vem da tabela Skins.**

Linha 20: FROM Jogadores

O FROM diz de onde vamos começar a ler os dados. O ponto de partida é a tabela principal de Jogadores.

Linha 21: INNER JOIN Skins ON Jogadores.id_aluno = Skins.id_dono;

ELE CRUZA A TABELA `JOGADORES` COM A TABELA `SKINS`.

O ON é a regra do cruzamento: "Só junte as linhas onde o código de identificação do jogador (id_aluno) for exatamente igual ao código do dono da skin (id_dono)".

O resultado prático: Essa lista só vai mostrar os jogadores que têm pelo menos uma skin. Se um aluno criou a conta no jogo, mas nunca ganhou ou comprou uma skin, ele fica totalmente de fora desse relatório. É um "clube exclusivo".

EXPLICANDO O QUE SIGNIFICA PALAVRA POR PALAVRA DENTRO DO LEFT JOIN

SEGUNDA CONSULTA: TODO MUNDO ENTRA

As linhas 24 e 25 são idênticas à consulta anterior (mesma busca, mesma tabela de origem). A grande diferença está na linha 26:

Linha 26: LEFT JOIN Skins ON Jogadores.id_aluno = Skins.id_dono;

O `LEFT JOIN` (junção à esquerda) é mais "inclusivo". Ele diz ao banco de dados: “traga **todos os registros da tabela da esquerda (que é a tabela `jogadores`, declarada no `from`), não importa o que aconteça".**

Ele também tenta cruzar com a tabela Skins usando a mesma regra do ON.

O resultado prático: Essa lista vai mostrar absolutamente todos os jogadores cadastrados. Se o jogador tiver uma skin, o nome dela aparece ao lado. Mas, se o jogador for free-to-play (não tiver nenhuma skin), o nome dele vai aparecer na lista mesmo assim, e na coluna da skin vai aparecer um espaço vazio chamado NULL (nulo).

RESUMINDO:

- **INNER JOIN:** Só mostra quem deu "match" nas duas tabelas. (Ex: só os alunos que têm skin na conta).
- **LEFT JOIN:** Prioriza a tabela da esquerda. Mostra todos os alunos da turma. Quem não tem skin fica com a coluna em branco (NULL), mas não é expulso da lista.

EXPLICANDO LINHA POR LINHA DO CÓDIGO (COMO VOCÊS PEDIRAM ANO PASSADO)

COMENTÁRIOS (Linhas 2 e 14)

- **Linhas com --:** Sempre que o código começa com dois tracinhos, é um **comentário**. O banco de dados ignora completamente essa linha. Serve só para os humanos se organizarem. Ok?

ESTRUTURA DA TABELA (Linhas de 3 a 6)

Aqui nós estamos criando a estrutura da tabela, o nosso esqueleto.

- **Linha 3: CREATE TABLE Jogadores (**
  Comando para criar uma nova tabela chamada "Jogadores". O parêntese aberto indica que vamos listar as colunas dela.
- **Linha 4: id_aluno INT PRIMARY KEY,**
  Criamos a primeira coluna.
  INT significa que ela só aceita números inteiros.
  PRIMARY KEY (Chave Primária) é o **CPF** do jogador no sistema**. É um número único e intransferível.** Ninguém no universo do jogo pode ter o mesmo número que outro jogador.
- **Linha 5: nick_name VARCHAR(50)**
  Criamos a coluna para o apelido. VARCHAR é o tipo de dado para texto (letras e números). O (50) é o limite: o jogador não pode escolher um nick com mais de 50 caracteres, senão o banco recusa.

CRIANDO A NOSSA TABELA DE SKINS (modificações visuais dos nossos personagens, roupas, armas, cabelos e afins)

- **Linha 8: CREATE TABLE Skins (**
  Criando a tabela para guardar as roupas e armas do jogo.

- **Linha 9: id_skin INT PRIMARY KEY,**
  O "código de barras" único da skin. Nenhuma outra skin pode ter esse número.
  Já falei ali em cima o que significa INT e PRIMARY KEY, basta ir pra página anterior.
- **Linha 10: nome_skin VARCHAR(50),**
  O nome do item, também limitado a 50 letras.
- **Linha 11: id_dono INT**
  Essa coluna vai guardar o "CPF" (o id_aluno) do jogador que comprou aquele item. É o gancho que vai conectar as duas tabelas mais tarde.

INSERINDO OS DADOS NO JOGO

As tabelas estavam vazias, correto? Agora vamos cadastrar o pessoal usando o comando INSERT INTO (literalmente é: INSERIR DENTRO).

- **Linha 15: Cadastrando os Jogadores**
  - VALUES (1, 'NinjaPro'): Cadastramos o aluno de CPF 1, que escolheu o nick 'NinjaPro'.
  - Depois cadastramos o número 2 ('CatPlayer') e o número 3 ('Ghost'). Temos 3 jogadores no servidor.
- **Linha 16: Dando os itens para os jogadores**
  - (10, 'Capa de Fogo', 1): Criamos a skin de código 10, chamada 'Capa de Fogo'. E quem é o dono? O jogador número 1 (o NinjaPro).
  - (11, 'Espada Laser', 2): Criamos a skin número 11, 'Espada Laser', e demos para o dono 2 (o CatPlayer).
  - *Atenção:* O jogador 3, o 'Ghost', não comprou nenhuma skin.

RESULTADOS DAS BUSCAS

- **O INNER JOIN (Linhas 19 a 21):** Lembra que ele era o "clube exclusivo"? O resultado dessa busca na tela vai ser apenas:
  NinjaPro | Capa de Fogo
  CatPlayer | Espada Laser

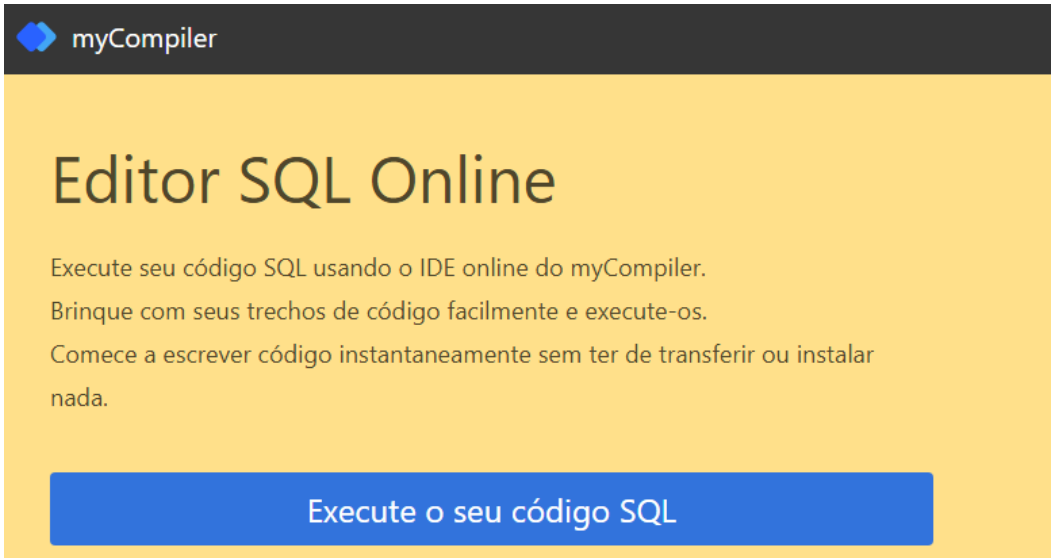
- *(O coitado do Ghost foi barrado da festa porque o sistema não achou nenhuma skin cruzando com o ID dele).*

- **O LEFT JOIN (Linhas 24 a 26):** Lembra que ele era o "todo mundo entra"? O resultado na tela vai ser:
  NinjaPro | Capa de Fogo
  CatPlayer | Espada Laser
  Ghost | NULL *(O Ghost aparece na lista geral, mas como ele não tem itens, o banco de dados escreve "null" - que significa nulo/vazio - na coluna da skin). Ele não deixa de aparecer, diferente do INNER JOIN que lá não aparece. No LEFT JOIN ELE APARECE, mesmo tedno seu valor nulo.*

PARA QUEM NÃO CONSEGUE INSTALAR O BANCO DE DADOS NO PC DE CASA (PC DA XUXA, TO BRINCANDO)

Acessem o site:

1) https://www.mycompiler.io/pt/online-sql-editor
2) Cliquem em "executar o seu código SQL"



3) Coloquem o código no espaço abaixo do nome SQL e logo depois cliquem em "executar" e vejam o código funcionar (ou não):

myCompiler Português Recentes Entrar Inscreva-se

Digite um título...

SQL Executar Salvar

```sql
-- Criando a tabela de jogadores (Lado ESQUERDO)
CREATE TABLE Jogadores (
    id_jogador INT PRIMARY KEY,
    nick_name VARCHAR(50)
);

-- Criando a tabela de Skins compradas (Lado DIREITO)
CREATE TABLE Skins (
    id_skin INT PRIMARY KEY,
    nome_skin VARCHAR(50),
```

Saída

```
NinjaPro|Capa de Fogo
CatPlayer|Espada Laser
NinjaPro|Capa de Fogo
CatPlayer|Espada Laser
Ghost|

[Execution complete with exit code 0]
```

EXERCÍCIO 2: LOJA DE APLICATIVOS E GAMES (OTÁVIOSTORE)

Temos três tabelas:

1) Jogadores: Cadastro de quem usa a plataforma.

2) Jogos: Catálogo de jogos disponíveis na loja.

3) Compras: O registro de quem comprou o quê.

SQL

```sql
-- Criando as tabelas
CREATE TABLE Jogadores (
    id_jogador INT PRIMARY KEY,
    nome_usuario VARCHAR(50)
);

CREATE TABLE Jogos (
    id_jogo INT PRIMARY KEY,
    titulo VARCHAR(50),
    preco DECIMAL(10,2)
);

CREATE TABLE Compras (
    id_compra INT PRIMARY KEY,
    id_jogador_fk INT,
    id_jogo_fk INT
);

-- Inserindo dados
INSERT INTO Jogadores VALUES (1, 'Paula_Gamer'), (2, 'Rafael_Tech'), (3, 'Rodrigo_Explorador');

INSERT INTO Jogos VALUES (10, 'Cyber Adventure', 150.00), (20, 'Space War', 80.00), (30, 'Medieval Quest', 120.00);

-- Paula comprou Cyber Adventure (10)
-- Rafael comprou Space War (20) e Medieval Quest (30)
-- Rodrigo não comprou nada ainda.
INSERT INTO Compras VALUES (101, 1, 10), (102, 2, 20), (103, 2, 30);
```

AGORA VEM O DESAFIO PARA VOCÊS:

Vocês terão que fazer 3 selects utilizando left join e/ou inner join:

- Relatório de vendas primário: listem o nome do jogador e o título do jogo para todas as compras realizadas (apenas quem comprou algo).
- Engajamento de usuários: listem todos os jogadores cadastrados e os títulos dos jogos que eles compraram. Se o jogador não comprou nada, o nome do jogo deve aparecer como NULL.
- Inventário da loja: listem todos os jogos da loja e o nome de quem os comprou. Se um jogo nunca foi vendido, o nome do comprador deve aparecer como NULL.

AMBOS OS EXERCÍCIOS VALEM MENÇÃO. ASSIM QUE TERMINAREM, ME MOSTREM.